
PERFREASONING: How Well Do LLMs Reason on Hardware Performance?

Dan Zhao, Karthikeyan Sankaralingam, Christos Kozyrakis, Qijing Huang

NVIDIA

Abstract

Performance modeling is central to hardware design and software optimization, yet constructing these models requires structured reasoning about computation, data reuse, storage, and movement. We introduce PERFREASONING, a benchmark that evaluates LLMs both as direct performance reasoners and as generators of analytical performance-model code. Given workload, architecture, and mapping specifications, models compare mappings and predict off-chip traffic and buffer requirements. The strongest closed-source models exceed 90% on reasoning-based Q&A, and the best open-weight model reaches 82.4%. However, model construction is substantially harder: while GPT-5.6 Sol exceeds 80% pass rate, all other model configurations average below 15% and vary markedly across runs. Task-specific RL raises a 4B model’s mapping-reasoning accuracy by 15.7 points, whereas feedback-free multi-round self-revision prompting is not reliably effective. PERFREASONING exposes the gap between plausible architectural reasoning and reliable performance-model construction. We will publicly release the benchmark to support reproducible evaluation and track future progress.

1 Introduction

Performance models guide hardware design and software optimization before expensive simulation or implementation [Chen et al. \[2018\]](#), [Zheng et al. \[2020\]](#), [Parashar et al. \[2019\]](#). Given a workload, an architecture, and a mapping, an analytical model estimates quantities such as memory traffic, storage utilization, execution time, energy, and cost. These models are typically developed manually by domain experts [Gober et al. \[2022\]](#), [Binkert et al. \[2011\]](#), [Lowe-Power et al. \[2020\]](#), [Kim et al. \[2016\]](#), [Jung et al. \[2015\]](#), [Won et al. \[2023\]](#), [Raj et al. \[2025\]](#), [Parashar et al. \[2019\]](#), [Wilton and Jouppi \[1996\]](#), [Muralimanohar et al. \[2007\]](#). Learned predictors reduce manual effort, but require trusted data and may generalize poorly outside their training distribution [Renda et al. \[2020\]](#), [Kumar et al. \[2021\]](#).

Frontier LLMs offer two complementary paths: reasoning directly about a design’s performance, or generating analytical performance-model code from specifications. Both require the model to connect tiling, loop order, and tensor placement to parallelism, reuse, storage, and traffic, then relate those demands to compute, memory, and interconnect constraints. Errors can silently steer an optimization loop toward poor decisions, especially when measurements are unavailable. We therefore ask: *Can LLMs reliably reason about performance and build analytical performance models?*

We answer this question with PERFREASONING, which couples reasoning-based Q&A with execution-based evaluation of generated models. This paired design separates performance understanding from the ability to turn that understanding into quantitatively reliable code. Across recent open-weight and closed-source models, we find strong comparative reasoning but a large and unstable gap in model construction. Moreover, code-induced mapping rankings closely match direct LLM comparisons on average, showing that generated programs can preserve useful ordinal signal even when their absolute predictions are wrong.

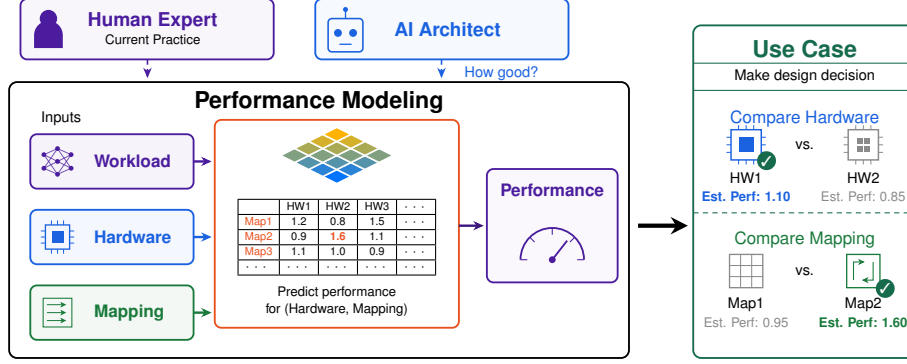


Figure 1: PERFREASONING evaluates LLMs as direct performance reasoners and as generators of analytical performance-model code.

Prior AI benchmarks span systems code generation [Ouyang et al. \[2025\]](#), [Wen et al. \[2025\]](#), [Wang et al. \[2026b\]](#), [Lin et al. \[2026\]](#), HLS and RTL [Abi-Karam and Hao \[2025\]](#), [Khan et al. \[2026\]](#), [Liu et al. \[2023\]](#), [Lu et al. \[2023\]](#), [Liu et al. \[2025\]](#), [Pinckney et al. \[2025\]](#), and chip and physical design [Yu et al. \[2026\]](#), [Li et al. \[2026\]](#); ArchEval evaluates broader tool-using architecture agents [Wang et al. \[2026a\]](#). Meanwhile, learning-based design-space exploration and emerging AI-architect flows depend on trusted evaluators to score candidate designs [Krishnan et al. \[2023\]](#), [Samajdar et al. \[2021\]](#), [Architecture 2.0 Contributors \[2026\]](#), [Sankaralingam \[2026\]](#). PERFREASONING complements these efforts by isolating performance reasoning and analytical performance-model construction.

Contributions.

- We introduce PERFREASONING, evaluating LLMs as performance reasoners and generators of analytical performance model code.
- We evaluate open-weight and closed-source frontier models, revealing a large reasoning–construction gap and substantial run-to-run variance.
- We test task-specific RL and feedback-free multi-round self-revision: RL adds 15.7 points, whereas self-revision is unreliable.

2 Benchmark

Tasks. Each Q&A task in the benchmark presents two mappings using three prompt templates agreement with a correct statement, agreement with its negation, and a direct comparison asking which mapping is better. We report the minimum accuracy across formulations to control for agreement bias. Each pair isolates one mapping axis—tile size, loop order, or tensor keep/bypass—and compares off-chip traffic or buffer capacity, key inputs to analytical performance and energy models.

The construction task prompts each LLM to generate one Python analytical model. The program reads workload, architecture, and mapping YAML specifications and predicts total and per-tensor backing-store accesses and buffer capacity. We run the generated program on hidden cases and compare it with Timeloop-derived labels [Parashar et al. \[2019\]](#). A case passes when both access and capacity predictions agree within 10^{-6} relative error; we also report Q-error and pairwise preference accuracy. The latter asks only whether the generated model ranks two mappings in the same order as ground truth. ranks two mappings in the same order as the reference model.

Coverage and Setup. The benchmark contains 216 valid configurations: 108 paired mappings spanning matrix multiplication, batched matrix multiplication, and convolution. The fixed two-level architecture is MainMemory $\rightarrow$ Buffer $\rightarrow$ MACC; mappings vary temporal tiling, loop permutation, and tensor retention. Labels span more than seven orders of magnitude in memory traffic and include both all-kept and bypass configurations. We evaluate 16 model families released or available from April-July 2026, including Claude, DeepSeek, Gemini, GLM, GPT, GPT-OSS, and MiniMax variants, at each provider-supported reasoning effort. We sample repeated generations when available and execute each program in an isolated environment. Appendix A gives full task and setup details.

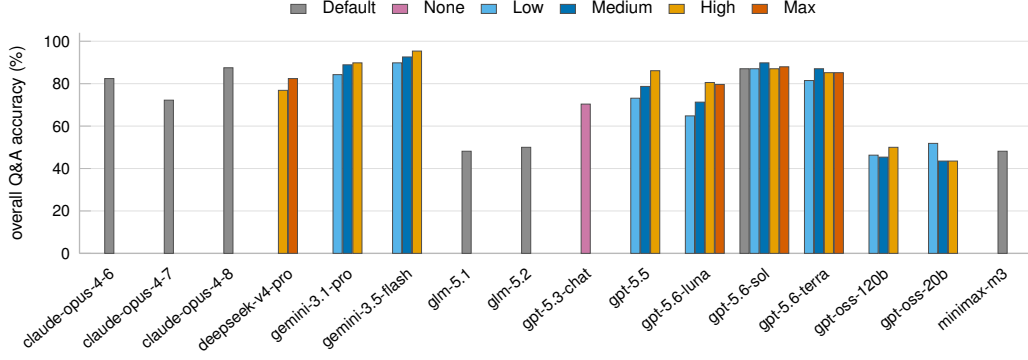


Figure 2: Q&A accuracy by model and reasoning effort. Strong closed models exceed 90%; the best open-weight result is 82.4%.

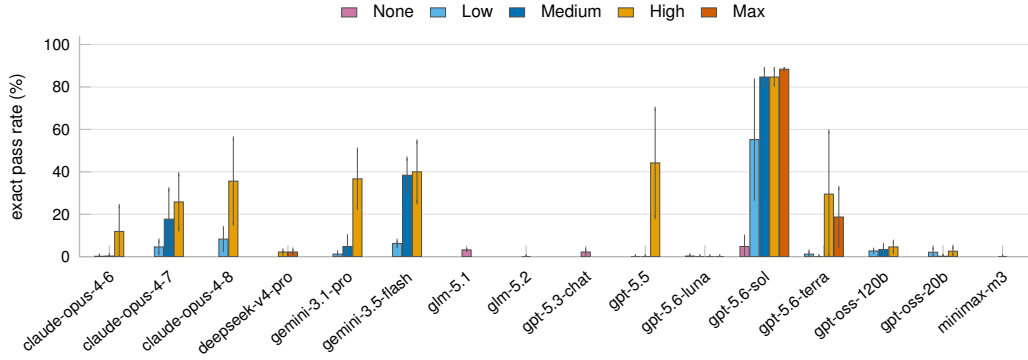


Figure 3: Single-shot pass rate for generated analytical models. Bars show means and error bars show variation across repeated generations.

3 Results

Frontier models reason well about mappings. Figure 2 shows that the strongest closed-source configurations reach 92.6–95.4% Q&A accuracy, while DeepSeek-V4-Pro, the strongest open-weight model, reaches 82.4%. Loop order is the most discriminative mapping axis: strong models distinguish reuse-sensitive permutations, whereas weaker models often treat them as equivalent. The appendix reports per-axis, workload, historical, and fine-tuning breakdowns.

Constructing a reliable model is much harder. Figure 3 reveals a sharp separation from Q&A. GPT-5.6 Sol is the only family with consistently high performance, reaching 84.7–88.4% at medium through max effort. Every other model/effort configuration averages below 45%, and their aggregate average is below 15%. Buffer capacity is structurally easier than memory-access accounting; incorrect access predictions can miss by many orders of magnitude. Repeated samples also vary substantially, so a plausible program from one run does not imply a reliable evaluator.

Code preserves rankings, not necessarily values. Direct Q&A and generated-model ranking evaluate the same ordinal question: which mapping moves less data? Across 31 aligned model/effort configurations, direct comparison and code-induced ranking average 78.3% and 78.5%, respectively; code is higher in 15 configurations and lower in 16. Thus, translation into code preserves the comparative signal on average but does not systematically improve it. This is useful for selecting between candidates, but it cannot establish trustworthy absolute traffic or buffer estimates.

Task-specific RL improves mapping reasoning. Figure 5 compares the same Qwen3-4B backbone before and after verifier-guided RL on binary mapping questions. Accuracy rises from 54.3% to 70.0%, showing that additional domain training can improve the reasoning prerequisite, although its effect on full model construction remains open.

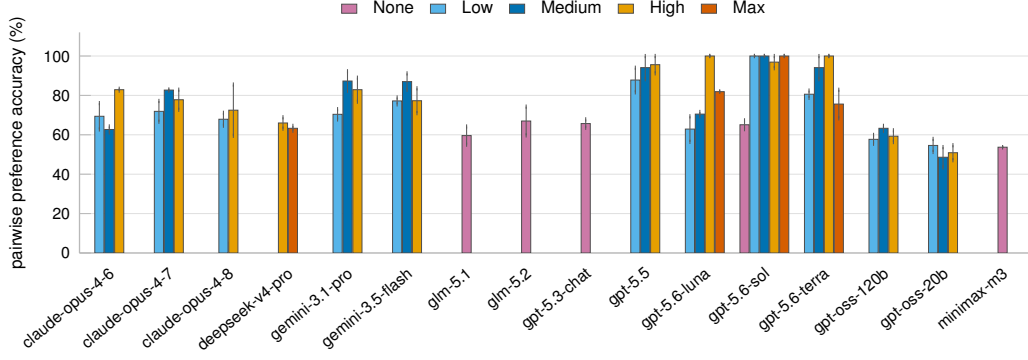


Figure 4: Mapping-ranking accuracy induced by generated analytical models. Programs often preserve the correct ordering despite inaccurate values.

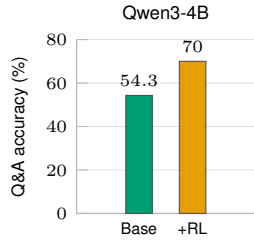


Figure 5: Verifier-guided RL improves Qwen3-4B mapping-reasoning accuracy from 54.3% to 70.0%.

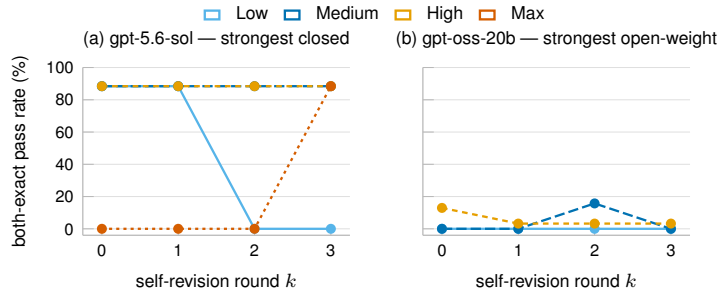


Figure 6: Feedback-free multi-round self-revision is inconsistent for the strongest closed and open-weight models.

Multi-round self-revision is not reliably effective. We prompt each model to inspect and revise its performance-model code for up to three rounds without execution or pass/fail feedback. Figure 6 shows representative trajectories; across all 41 configurations, 18 never change, 10 finish worse, and only 7 finish better. Mean pass rate rises from 14.0% to just 16.7%, showing that models cannot reliably recognize and retain improvements. Full results are reported in Appendix H.

Implications and limitations. The best frontier models exceed 90% accuracy when reasoning about tiling, data reuse, and placement effects, making them promising for agentic design loops. However, variability across runs requires validation before their outputs can serve as reliable evaluators.

Our benchmark evaluates frontier models out of the box through a natural-language interface and does not establish how complex a performance model frontier models can construct. Formal verification tools, as used by Aristotle [Achim et al. \[2025\]](#), may enable more complex and reliable performance-model construction; we leave this to future work. Finally, Timeloop-derived labels measure analytical agreement rather than silicon accuracy; broader architectures and hardware-validated labels remain future work.

4 Conclusion

We introduced PERFREASONING, a benchmark of frontier LLMs as hardware-performance reasoners and builders of analytical model code. The strongest models reason accurately about mapping effects, yet only one evaluated configuration builds reliable models, and outcomes vary substantially across runs. Task-specific RL improves small-model reasoning, whereas feedback-free self-revision is unreliable. Progress toward AI-assisted chip design requires stronger reasoning and stable, accurate, and verifiable model generation. By separating reasoning from model construction, PERFREASONING provides a concrete benchmark for tracking that progress.

References

- Stefan Abi-Karam and Cong Hao. HLS-Eval: A benchmark and framework for evaluating LLMs on high-level synthesis design tasks. *arXiv preprint arXiv:2504.12268*, 2025. URL <https://arxiv.org/abs/2504.12268>.
- Tudor Achim, Alex Best, Alberto Bietti, Kevin Der, Mathis Fédérico, Sergei Gukov, Daniel Halpern-Leistner, Kirsten Henningsgard, Yury Kudryashov, Alexander Meiburg, Martin Michelsen, Riley Patterson, Eric Rodriguez, Laura Scharff, Vikram Shanker, Vladmir Sicca, Hari Sowrirajan, Aidan Swope, Matyas Tamas, Vlad Tenev, Jonathan Thomm, Harold Williams, and Lawrence Wu. Aristotle: IMO-level automated theorem proving, 2025. URL <https://arxiv.org/abs/2510.01346>.
- Architecture 2.0 Contributors. Architecture 2.0: AI-Assisted Computer Architecture. <https://arch2.mlsysbook.ai/>, 2026. Accessed: 2026-07-21.
- Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh Sardashti, Rathijit Sen, Korey Sewell, Muhammad Shoaib, Nilay Vaish, Mark D. Hill, and David A. Wood. The gem5 simulator. *ACM SIGARCH Computer Architecture News*, 39(2):1–7, May 2011. doi: 10.1145/2024716.2024718. URL <https://doi.org/10.1145/2024716.2024718>.
- Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: An automated End-to-End optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 578–594. USENIX Association, October 2018. ISBN 978-1-939133-08-3. URL <https://www.usenix.org/conference/osdi18/presentation/chen>.
- Nathan Gober, Gino Chacon, Lei Wang, Paul V. Gratz, Daniel A. Jiménez, Elvira Teran, Seth Pugsley, and Jinchun Kim. The championship simulator: Architectural simulation for education and competition. *arXiv preprint arXiv:2210.14324*, 2022. doi: 10.48550/arXiv.2210.14324. URL <https://arxiv.org/abs/2210.14324>.
- Matthias Jung, Christian Weis, and Norbert Wehn. DRAMSys: A flexible DRAM subsystem design space exploration framework. *IPSJ Transactions on System LSI Design Methodology*, 8:63–74, 2015. doi: 10.2197/ipsjtsldm.8.63. URL <https://doi.org/10.2197/ipsjtsldm.8.63>.
- M. Zafir Sadik Khan, Kimia Azar, and Hadi Kamali. Bench4HLS: End-to-end evaluation of LLMs in high-level synthesis code generation. *arXiv preprint arXiv:2601.19941*, 2026. URL <https://arxiv.org/abs/2601.19941>.
- Yoongu Kim, Weikun Yang, and Onur Mutlu. Ramulator: A fast and extensible DRAM simulator. *IEEE Computer Architecture Letters*, 15(1):45–49, January 2016. doi: 10.1109/LCA.2015.2414456. URL <https://doi.org/10.1109/LCA.2015.2414456>.
- Srivatsan Krishnan, Amir Yazdanbakhsh, Shvetank Prakash, Jason Jabbour, Ikechukwu Uchendu, Susobhan Ghosh, Behzad Boroujerdian, Daniel Richins, Devashree Tripathy, Aleksandra Faust, and Vijay Janapa Reddi. ArchGym: An open-source gymnasium for machine learning assisted architecture design. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023. doi: 10.1145/3579371.3589049.
- Aviral Kumar, Amir Yazdanbakhsh, Milad Hashemi, Kevin Swersky, and Sergey Levine. Data-driven offline optimization for architecting hardware accelerators. *arXiv preprint arXiv:2110.11346*, 2021.
- Beibin Li, Yao Lu, Chi Wang, and Srikanth Kandula. Q-error bounds of random uniform sampling for cardinality estimation, 2021.
- Qiufeng Li, Rongqian Chen, Quan Cheng, Chengxuan Wang, Sizhe Tang, Wuxi Li, Duo Ding, Chia-Tung Ho, Haoxing Ren, David Z. Pan, Tian Lan, and Weidong Cao. PDAGENT-BENCH: Characterizing, grounding, and architecting LLM agents for VLSI physical design. *arXiv preprint arXiv:2606.17253*, 2026. URL <https://arxiv.org/abs/2606.17253>.

- Edward Lin, Sahil Modi, Siva Kumar Sastry Hari, Qijing Huang, Zhifan Ye, Nestor Qin, Fengzhe Zhou, Yuan Zhang, Jingquan Wang, Sana Damani, Dheeraj Peri, Ouye Xie, Aditya Kane, Moshe Maor, Michael Behar, Triston Cao, Rishabh Mehta, Vartika Singh, Vikram Sharma Mailthody, Terry Chen, Zihao Ye, Hanfeng Chen, Tianqi Chen, Vinod Grover, Wei Chen, Wei Liu, Eric Chung, Luis Ceze, Roger Bringmann, Cyril Zeller, Michael Lightstone, Christos Kozyrakis, and Humphrey Shi. SOL-ExecBench: Speed-of-light benchmarking for real-world GPU kernels against hardware limits. *arXiv preprint arXiv:2603.19173*, 2026. URL <https://arxiv.org/abs/2603.19173>.
- Mingjie Liu, Nathaniel Pinckney, Brucec Khailany, and Haoxing Ren. VerilogEval: Evaluating large language models for verilog code generation. *arXiv preprint arXiv:2309.07544*, 2023. URL <https://arxiv.org/abs/2309.07544>.
- Shang Liu, Yao Lu, Wenji Fang, Mengming Li, and Zhiyao Xie. OpenLLM-RTL: Open dataset and benchmark for LLM-aided design RTL generation. *arXiv preprint arXiv:2503.15112*, 2025. URL <https://arxiv.org/abs/2503.15112>.
- Jason Lowe-Power et al. The gem5 simulator: Version 20.0+. *arXiv preprint arXiv:2007.03152*, 2020. doi: 10.48550/arXiv.2007.03152. URL <https://arxiv.org/abs/2007.03152>.
- Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. RTLLM: An open-source benchmark for design RTL generation with large language model. *arXiv preprint arXiv:2308.05345*, 2023. URL <https://arxiv.org/abs/2308.05345>.
- Naveen Muralimanohar, Rajeev Balasubramonian, and Norman P. Jouppi. Optimizing NUCA organizations and wiring alternatives for large caches with CACTI 6.0. In *40th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 3–14. IEEE, December 2007. doi: 10.1109/MICRO.2007.33. URL <https://doi.org/10.1109/MICRO.2007.33>.
- Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs write efficient GPU kernels? *arXiv preprint arXiv:2502.10517*, 2025. URL <https://arxiv.org/abs/2502.10517>.
- Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A. Ying, Anurag Mukkara, Rangharajan Venkatesan, Brucec Khailany, Stephen W. Keckler, and Joel Emer. Timeloop: A systematic approach to DNN accelerator evaluation. In *2019 IEEE International Symposium on Performance Analysis of Systems and Software*, pages 304–315. IEEE, 2019. doi: 10.1109/ISPASS.2019.00042. URL <https://doi.org/10.1109/ISPASS.2019.00042>.
- Nathaniel Pinckney, Chenhui Deng, Chia-Tung Ho, Yun-Da Tsai, Mingjie Liu, Wenfei Zhou, Brucec Khailany, and Haoxing Ren. Comprehensive verilog design problems: A next-generation benchmark dataset for evaluating large language models and agents on RTL design and verification. *arXiv preprint arXiv:2506.14074*, 2025. URL <https://arxiv.org/abs/2506.14074>.
- Ritik Raj, Sarbartha Banerjee, Nikhil Chandra, Zishen Wan, Jianming Tong, Ananda Samajdar, and Tushar Krishna. SCALE-Sim v3: A modular cycle-accurate systolic accelerator simulator for end-to-end system analysis. In *2025 IEEE International Symposium on Performance Analysis of Systems and Software*, pages 186–200. IEEE, 2025. doi: 10.1109/ISPASS64960.2025.00026. URL <https://doi.org/10.1109/ISPASS64960.2025.00026>.
- Alex Renda, Yishen Chen, Charith Mendis, and Michael Carbin. DiffTune: Optimizing cpu simulator parameters with learned differentiable surrogates. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 442–455. IEEE, 2020.
- Ananda Samajdar, Jan Moritz Joseph, Matthew Denton, and Tushar Krishna. AIRCHITECT: Learning custom architecture design and mapping space. *arXiv preprint arXiv:2108.08295*, 2021. URL <https://arxiv.org/abs/2108.08295>.
- Karthikeyan Sankaralingam. Computer architecture’s AlphaZero moment: Automated discovery in an encircled world. *arXiv preprint arXiv:2604.03312*, 2026. URL <https://arxiv.org/abs/2604.03312>.

- Chenyu Wang, Zishen Wan, Jeffrey Ma, Shvetank Prakash, Zhenting Qi, Haebin Do, Andy Cheng, Arya Tschand, Jiahe Shi, Yilun Du, and Vijay Janapa Reddi. ArchEval: Measuring AI agents as computer architects. *arXiv preprint arXiv:2607.03601*, 2026a. doi: 10.48550/arXiv.2607.03601. URL <https://arxiv.org/abs/2607.03601>.
- Han Wang, Jintao Zhang, Kai Jiang, Haoxu Wang, Jianfei Chen, and Jun Zhu. KernelBench-X: A comprehensive benchmark for evaluating LLM-generated GPU kernels. *arXiv preprint arXiv:2605.04956*, 2026b. URL <https://arxiv.org/abs/2605.04956>.
- Zhongzhen Wen, Yinghui Zhang, Zhong Li, Zhongxin Liu, Linna Xie, and Tian Zhang. MultiKernelBench: A multi-platform benchmark for kernel generation. *arXiv preprint arXiv:2507.17773*, 2025. URL <https://arxiv.org/abs/2507.17773>.
- Steven J. E. Wilton and Norman P. Jouppi. CACTI: An enhanced cache access and cycle time model. *IEEE Journal of Solid-State Circuits*, 31(5):677–688, May 1996. doi: 10.1109/4.509850. URL <https://doi.org/10.1109/4.509850>.
- William Won, Taekyung Heo, Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. ASTRA-sim2.0: Modeling hierarchical networks and disaggregated systems for large-model training at scale. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software*, pages 283–294. IEEE, 2023. doi: 10.1109/ISPASS57527.2023.00035. URL <https://doi.org/10.1109/ISPASS57527.2023.00035>.
- Zhongkai Yu, Chenyang Zhou, Yichen Lin, Hejia Zhang, Haotian Ye, Junxia Cui, Zaifeng Pan, Jishen Zhao, and Yufei Ding. ChipBench: A next-step benchmark for evaluating LLM performance in ai-aided chip design. *arXiv preprint arXiv:2601.21448*, 2026. URL <https://arxiv.org/abs/2601.21448>.
- Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. Ansor: Generating High-Performance tensor programs for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 863–879. USENIX Association, November 2020. ISBN 978-1-939133-19-9. URL <https://www.usenix.org/conference/osdi20/presentation/zheng>.

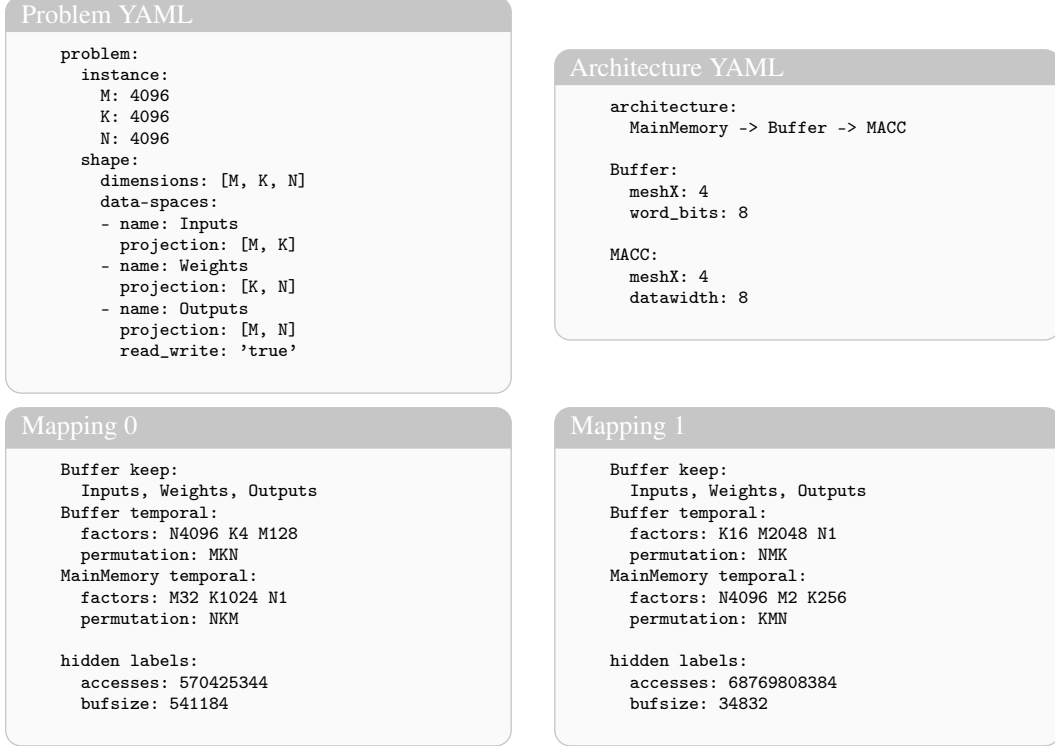


Figure 7: Example benchmark case. The generated program receives the problem, architecture, and mapping YAML files, but not the hidden labels shown here. The two mappings implement the same 4096^3 matrix multiplication but induce very different Timeloop-derived traffic and buffer requirements: mapping 0 has $120.6\times$ lower MainMemory accesses, while mapping 1 uses $15.5\times$ less Buffer capacity.

A Benchmark and Evaluation Details

Each generated program receives exactly three files: `prob.yaml`, which defines the tensor problem; `arch.yaml`, which defines the two-level hardware hierarchy; and `map.yaml`, which specifies temporal tiling, loop order, and keep/bypass decisions. The hidden `access.yaml` contains the reference result. The 216 cases are balanced across matrix multiplication, batched matrix multiplication, and convolution. Timeloop ground-truth access counts range from 98,304 to 4,389,456,576,512, and required buffer capacity ranges from 0 to 270,532,608 elements.

For Q&A, the benchmark contains 108 mapping pairs in a balanced 3×3 design over workload family and changed mapping axis. Each pair appears in three formulations. For model construction, each output must parse unseen YAML cases rather than hard-code benchmark answers. We record compile and runtime failures, exact access and capacity agreement, per-output Q-error, and paired ranking accuracy. Results are aggregated by model and reasoning-effort setting; error bars denote variation across repeated generated programs.

B Benchmark Case Example and Components

Figure 7 shows one complete benchmark case as the generated program sees it, together with the hidden ground truth. generated program sees it, together with the hidden reference labels. Table 1 summarizes the two components of the benchmark and what each is designed to test.

C Metric Definitions

This appendix gives the formal definitions of the metrics summarized in Section 2.

Table 1: Overview of the two benchmark components. QA-understanding tests qualitative mapping intuition, while performance modeling evaluates quantitative prediction ability.

Component	Task	Purpose	Stylized example
Q&A Mapping Understanding	Given two mappings for the same tensor program, answer questions about their relative MainMemory access behavior.	Tests whether the model understands tiling, loop order, reuse, memory hierarchy placement, and keep annotations, independent of detailed numeric prediction.	Map A: tile M,K Map B: tile N,K Which has fewer MainMemory accesses?
Analytical Performance Modeling	Given a tensor program and mapping, predict quantitative performance-model quantities such as memory accesses, buffer sizes, or ratios.	Evaluates whether the model can translate mapping understanding into accurate quantitative estimates; errors may include conceptual, arithmetic, or calibration failures.	MatMul: MK,KN→MN Mapping: tile M=64,K=8 Predict MainMemory accesses.

We use a strict exact-match metric (within a small error tolerance) as our main measure, i.e., the overall pass rate: a case passes only when the generated performance model predicts both total MainMemory accesses and required Buffer capacity correctly, up to a small numerical tolerance. This captures whether the synthesized model fully matches the Timeloop-derived analytical result, rather than only getting one part of the memory hierarchy right.

We also report metric-specific exact rates for accesses and bufsize separately. Accesses exactness reflects whether the model captures data movement, reuse, bypass behavior, and loop-order effects, while bufsize exactness reflects whether it correctly computes the resident on-chip tile footprint. We use q-error to measure how far predictions are from ground truth on a multiplicative scale. Since errors can span many orders of magnitude, we use this multiplicative metric to differentiate between small and large errors in predictions more clearly than absolute error. We also report metric-specific exact rates for accesses and bufsize separately. Accesses exactness reflects whether the model captures data movement, reuse, bypass behavior, and loop-order effects, while bufsize exactness reflects whether it correctly computes the resident on-chip tile footprint. We use q-error to measure how far predictions are from the reference labels on a multiplicative scale. Since errors can span many orders of magnitude, we use this multiplicative metric to differentiate between small and large errors in predictions more clearly than absolute error.

Let $y_{i,m}$ denote the Timeloop ground-truth value for case i and metric $m \in \{\text{accesses}, \text{bufsize}\}$, and let $\hat{y}_{i,m}$ denote the value predicted by the generated program. We treat a prediction as exact if it matches the ground truth within a small numerical tolerance, used exact if it matches the reference value within a small numerical tolerance, used only to avoid penalizing harmless integer-versus-floating-point formatting differences. The primary metric is both-exact pass rate:

$$\text{Pass}_{\text{both}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}_{i,\text{accesses}} \approx y_{i,\text{accesses}} \wedge \hat{y}_{i,\text{bufsize}} \approx y_{i,\text{bufsize}}],$$

where $N = 216$ in our benchmark. This metric is intentionally strict: a case is correct only if the model simultaneously predicts off-chip traffic and on-chip storage correctly. We also report per-metric exact pass rates:

$$\text{Pass}_m = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}_{i,m} \approx y_{i,m}], \quad m \in \{\text{accesses}, \text{bufsize}\}.$$

In addition to accuracy and pass rates, to better understand how off the model’s estimates are, we use a symmetric multiplicative error or q-error [Li et al. \[2021\]](#):

$$Q(\hat{y}, y) = \max\left(\frac{\max(\hat{y}, 1)}{\max(y, 1)}, \frac{\max(y, 1)}{\max(\hat{y}, 1)}\right).$$

A q-error of 1 is exact, a q-error of 2 means the estimate is off by a factor of two a q-error of 10 indicates the prediction is off from the ground truth by ten times, etc. Larger values indicate larger

Table 2: Pooled all-three Q&A accuracy (%) for each cell of the balanced 3×3 benchmark design (12 problems per cell).

Distinguishing axis	MatMul	Batched MM	Conv2D
Tiling	72.8	64.8	88.5
Loop order	71.5	68.0	69.3
Bypass	83.0	78.3	87.4

multiplicative error, potentially in either direction. The q-error provides additional information that overall pass rates and accuracy do not: for example, in the case where two models are both wrong for a given test case, one model can be worse at the prediction than the other (e.g., estimates that are twice off versus one that is 10^8 times off).

Finally, as each problem has two mappings, we also evaluate whether a generated performance model preserves the relative ordering of mappings by memory traffic so as to evaluate not just exact correctness but also its ability to rank mappings via said program. Let $\mathcal{P}$ be the set of mapping pairs with unequal true access counts. For a pair $(a, b) \in \mathcal{P}$, let A_a and A_b be the ground-truth accesses, and let $\hat{A}_a$ and $\hat{A}_b$ be the predicted accesses. Pairwise preference accuracy is:

$$\text{PrefAcc} = \frac{1}{|\mathcal{P}|} \sum_{(a,b) \in \mathcal{P}} \mathbf{1} \left[(\hat{A}_a < \hat{A}_b) = (A_a < A_b) \right].$$

This measures whether the generated model can identify the lower-traffic mapping, even when absolute traffic estimates are imperfect.

D Detailed Q&A Analysis

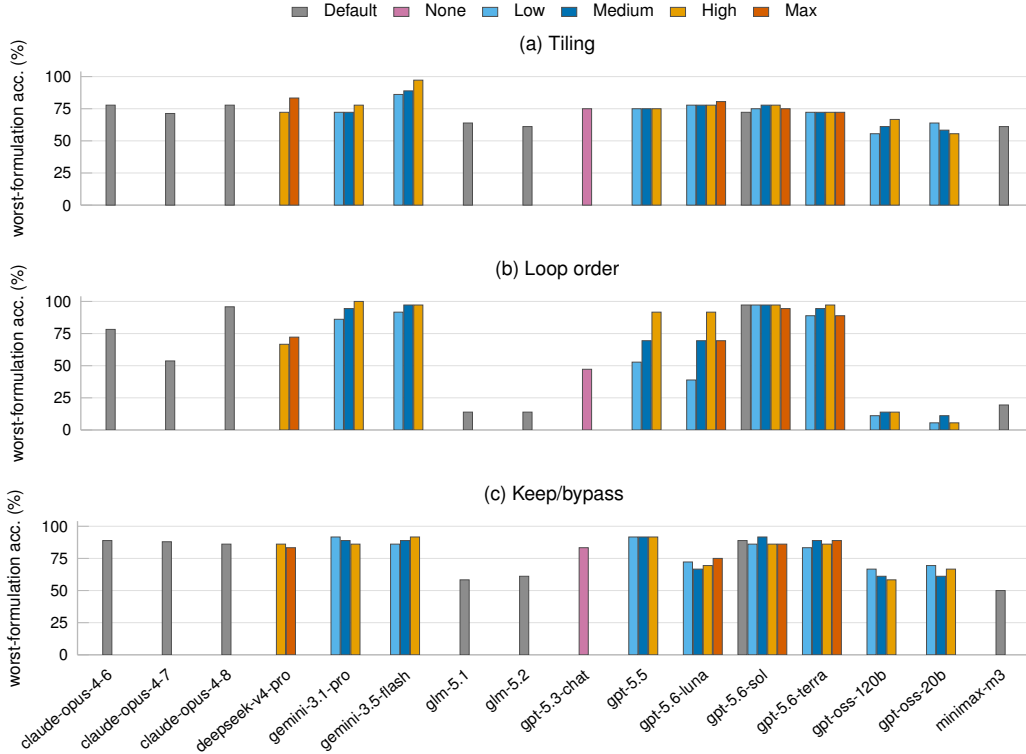


Figure 8: Worst-formulation Q&A accuracy by mapping axis. Loop-order sensitivity most clearly separates strong and weak configurations.

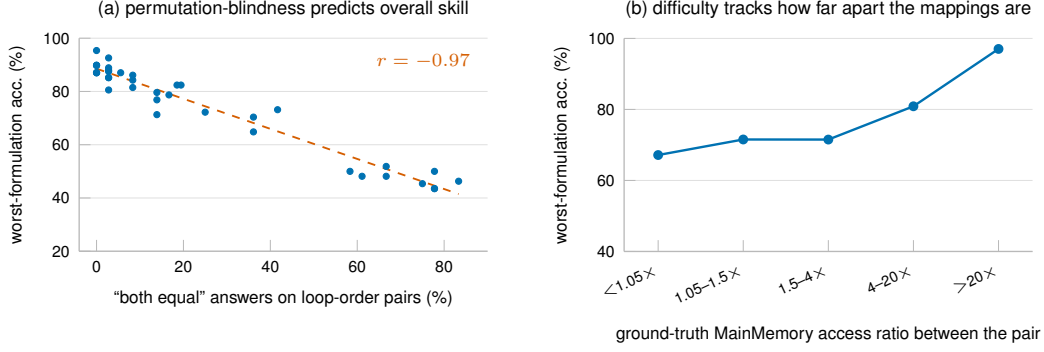


Figure 9: Loop-order permutation blindness: “equal” responses strongly predict loop-order errors and overall Q&A accuracy.

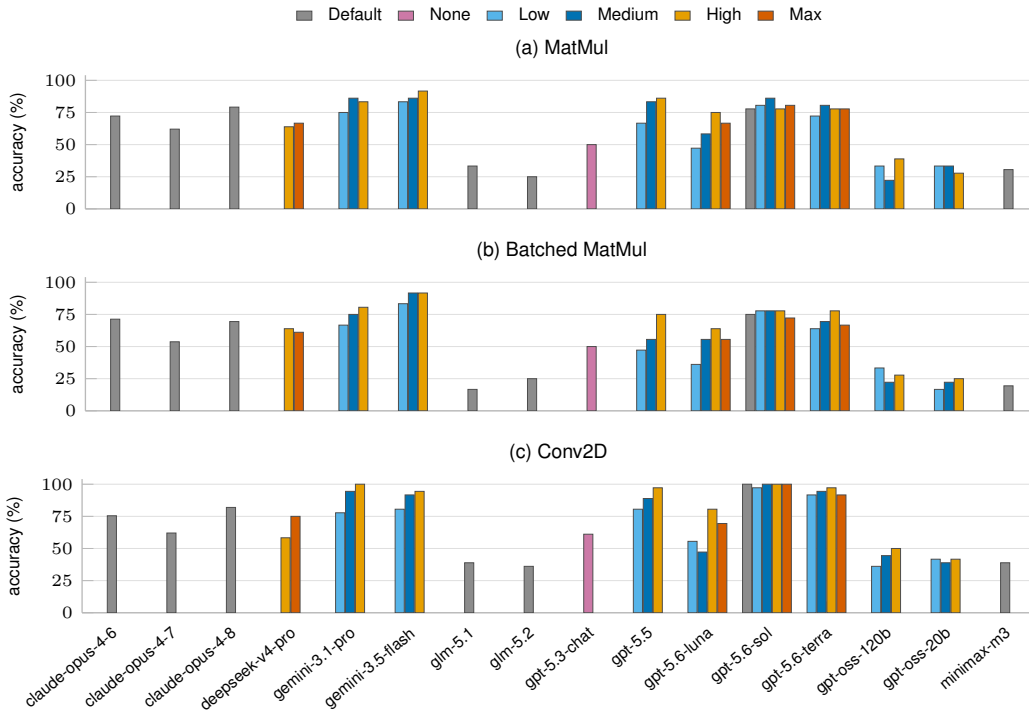


Figure 10: Worst-formulation Q&A accuracy by workload family.

Loop order is bimodal across model families: configurations either track the reuse consequences of permutation or frequently declare the mappings equal. Tiling and tensor retention vary more smoothly. Convolution is generally the easiest workload and batched matrix multiplication the hardest, but the mapping axis explains more of the cross-model separation than workload family.

E Progress and Domain-Specific Fine-Tuning

E.1 Progress Across Model Generations

Key Takeaway: Closed models now achieve near-perfect QA accuracy (95.4%), while the best open-weight model reaches 82.4%; both improve substantially over the 2025 snapshot, although a 13-point gap remains.

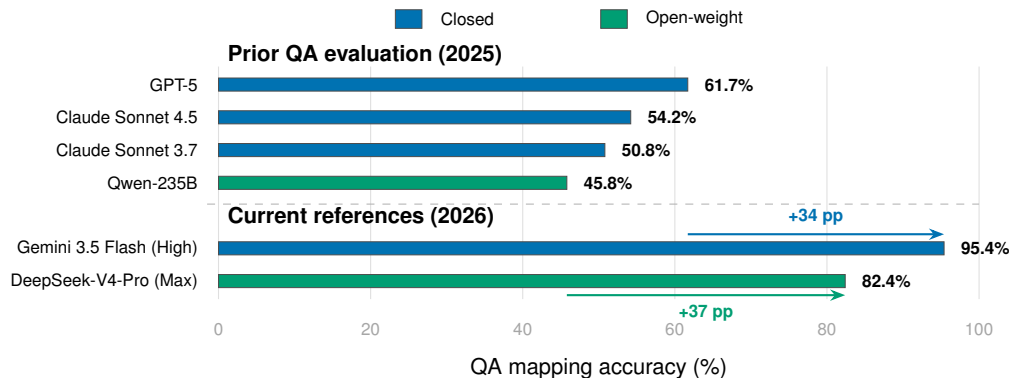


Figure 11: QA mapping-understanding results from the prior cohort and the strongest current references in Figure 2. Prior values use direct-comparison accuracy on 120 examples; current values use the stricter worst-formulation metric. We treat the closely aligned tasks as roughly comparable; the reported gains are therefore approximate.

To place the current results in context, we compare them with our prior QA evaluation cohort. The earlier evaluation used 120 direct mapping-comparison examples. GPT-5 (August 2025) answered 74 correctly (61.7%), followed by Claude Sonnet 4.5 (September 2025) at 54.2%, Claude Sonnet 3.7 (February 2025) at 50.8%, and the open-weight Qwen3-235B-A22B (April 2025) at 45.8%. Figure 11 contrasts these results with the strongest current references in Figure 2: Gemini 3.5 Flash (2026) reaches 95.4% at high reasoning effort, while DeepSeek-V4-Pro (April 2026) reaches 82.4% at max effort.

The strongest closed-model reference rises by approximately 34 percentage points, while the strongest general-purpose open-weight reference rises by approximately 37 points. Because the cohorts test closely aligned mapping-comparison tasks, we treat them as roughly comparable snapshots. The current worst-formulation metric is stricter, however, so these gains are approximate rather than a controlled longitudinal estimate.

E.2 RL Fine-Tuning Can Improve Mapping Reasoning

Key Takeaway: Additional task-specific training helps: verifier-guided RL on binary Q&A labels lifts a Qwen3-4B backbone from 54.3% to 70%, overtaking GPT-5 (August 2025) at 61.7%. Domain-specific fine-tuning can inject mapping expertise into small open-weight models.

The prior cohort also included our RL-fine-tuned Qwen3-4B (April 2025), shown in Figure 12. Verifier-guided RL on binary Q&A labels lifts the same backbone from 54.3% to 70% on the 120-example evaluation, past GPT-5 (August 2025) by 8.3 points and the far larger Qwen3-235B-A22B by 24.2. Mapping reasoning suits this setup: Timeloop-derived labels give objective rewards, and diverse loop orders, tiling factors, and keep/bypass choices expose exactly the reuse patterns on which general-purpose models fail.

The baseline is the same backbone without RL, so the 15.7-point gain is measured against a matched control, though from a single pass over the smaller legacy suite. The study also predates this evaluation by roughly nine months, and both closed-source and open-weight models have since improved substantially at architectural reasoning (95.4% and 82.4% today, Figure 11). The 70% is therefore not competitive now; what transfers is that binary Q&A labels alone lifted a small backbone past the best contemporaneous general-purpose model. Whether this specialization also improves traffic and buffer-size synthesis remains open.

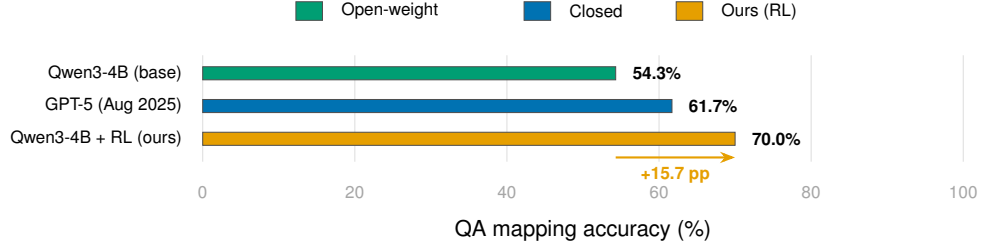


Figure 12: Verifier-guided RL fine-tuning on binary Q&A labels lifts a Qwen3-4B backbone from 54.3% to 70% on the 120-example QA evaluation, past GPT-5 (August 2025) at 61.7%. All three use the prior cohort’s direct-comparison metric and are therefore directly comparable.

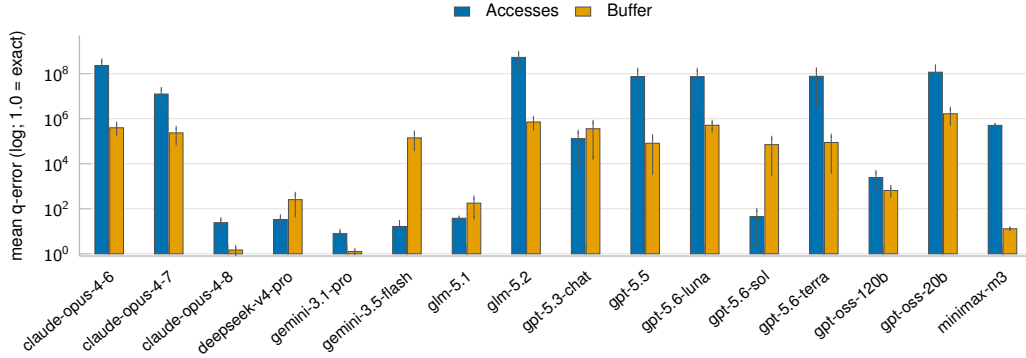


Figure 13: Mean Q-error for memory-access and buffer predictions; 1 is exact and lower is better.

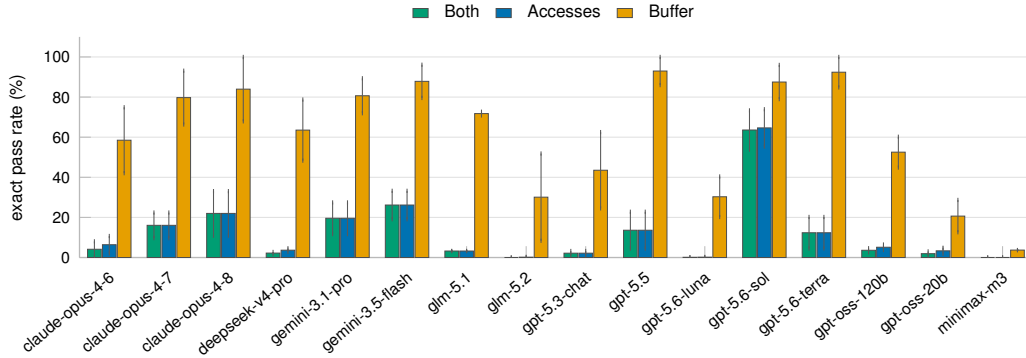


Figure 14: Exact-match decomposition. Buffer capacity is substantially easier than memory-access accounting for nearly every model family.

F Additional Performance-Model Results

High Q&A accuracy contains most of the strongest construction results, but the spread among high-Q&A models is large. Conceptual mapping understanding is therefore a useful diagnostic, not a sufficient certificate. The exact-match decomposition localizes much of the remaining gap to access accounting rather than capacity. Pairwise ranking is much stronger than absolute agreement, showing that systematic numerical errors can preserve the correct ordering.

G Reasoning Effort and Multi-Round Self-Revision

Each model may revise its initial program up to three times without execution or pass/fail feedback. Across 41 configurations, 18 remain unchanged through all rounds, 10 finish below their initial

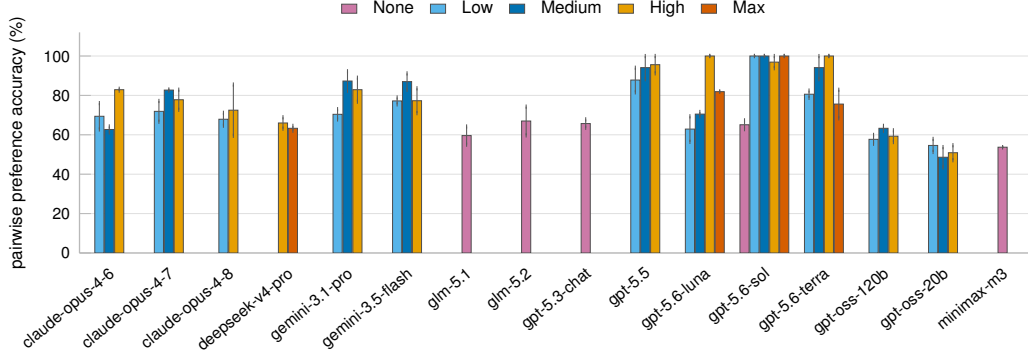


Figure 15: Pairwise preference accuracy induced by generated analytical models. Rankings can be correct even when absolute predictions are not.

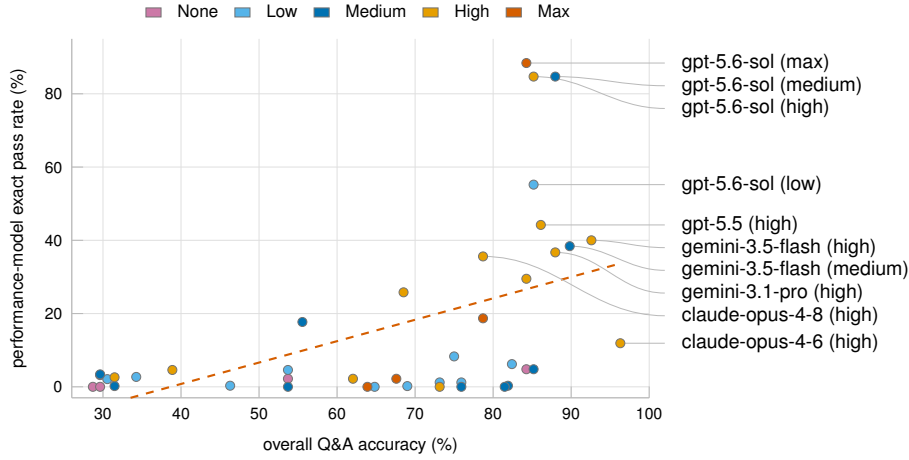


Figure 16: Q&A mapping understanding versus analytical-model pass rate across configurations ($r = 0.46$).

accuracy, and only 7 finish above it. Mean pass rate rises only from 14.0% to 16.7%, although selecting the best candidate in hindsight would reach 22.9%. Models can therefore generate a better candidate without reliably recognizing and retaining it.

H Per-Round Results for Multi-Round Self-Revision

Section 3 summarizes the effect of feedback-free multi-round self-revision on analytical performance modeling. Table 3 gives the full per-round breakdown behind that summary: for every (model, reasoning-effort) configuration, the both-exact pass rate of the program produced at each revision step k , together with the best rate achieved over all four candidates.

I Limitations and Extensions

The benchmark measures agreement with Timeloop and its accounting conventions, not physical accuracy on silicon. It fixes one two-level hierarchy and omits spatial tiling, deeper hierarchies, sparse and attention workloads, energy, latency, and area. Model versions changed during the evaluation window, providers expose reasoning effort differently, and replicate counts vary.

A natural extension is to evaluate measured GPUs, where DRAM traffic, occupancy, and kernel time replace tool-derived labels. Such an evaluation would require tolerance bands calibrated to measurement noise and controlled pairs that isolate coalescing, cache behavior, occupancy, and

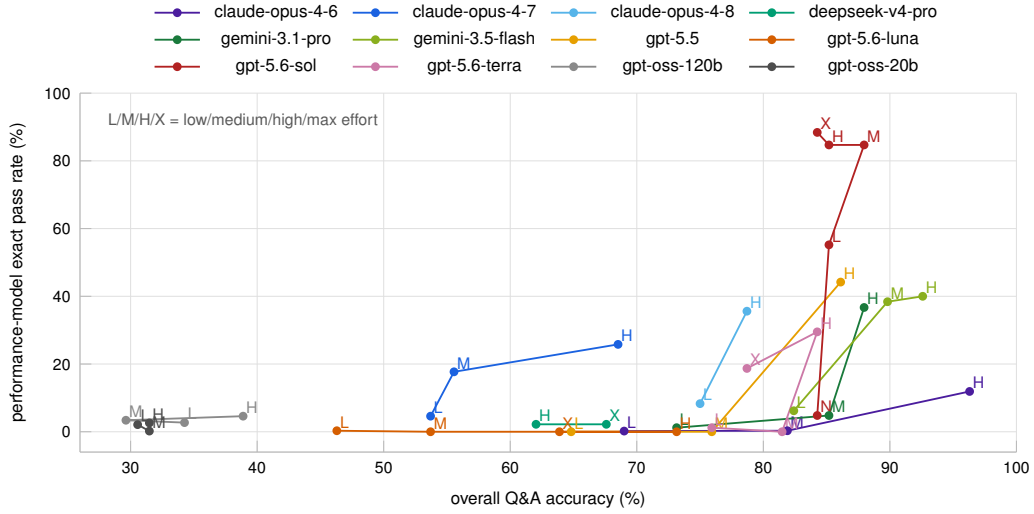


Figure 17: Q&A and analytical-model accuracy across reasoning-effort settings. Additional effort is not uniformly beneficial.

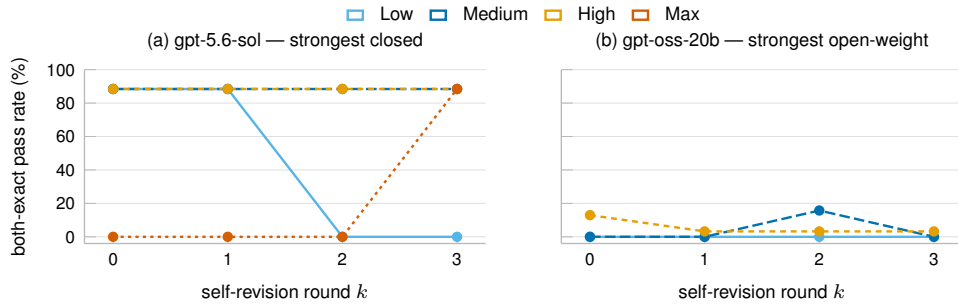


Figure 18: Feedback-free self-revision may help, harm, or leave accuracy unchanged for the strongest closed and open-weight models.

latency hiding. It would also test whether the controlled mapping-reasoning findings transfer from analytical ground truth to shipping hardware. from analytical reference labels to shipping hardware.

Table 3: **Both-exact pass rate (%) across three self-revision rounds on performance modeling.** Each row denotes one (model, reasoning-effort) configuration; the model generates a program at $k=0$ and is given the option to revise up to three times with no evaluation feedback.

Model	Effort	$k=0$	$k=1$	$k=2$	$k=3$	Best
opus-4.6	Low	0.0	0.0	0.0	0.0	0.0
	Medium	7.9	7.9	35.6	35.6	35.6
	High	0.0	0.0	0.0	0.0	0.0
opus-4.7	Low	11.1	13.9	13.9	13.9	13.9
	Medium	41.7	45.4	41.7	41.7	45.4
	High	25.9	35.6	35.6	35.6	35.6
opus-4.8	Low	18.5	18.5	18.5	18.5	18.5
	High	29.6	43.5	43.5	43.5	43.5
deepseek-v4-pro	High	3.2	0.0	0.0	0.0	3.2
	Max	3.2	3.2	0.0	3.2	3.2
gemini-3.1-pro	Low	3.7	3.2	3.2	3.2	3.7
	Medium	80.1	80.1	80.1	80.1	80.1
	High	22.5	22.5	22.5	1.2	22.5
gemini-3.5-flash	Low	7.4	7.4	7.4	0.0	7.4
	Medium	23.6	23.6	23.6	23.6	23.6
	High	3.7	3.7	3.7	35.6	35.6
glm-5.1	Default	0.0	0.0	0.0	0.0	0.0
glm-5.2	Default	0.0	0.0	0.0	0.0	0.0
gpt-5.3	Default	3.2	0.0	0.0	1.4	3.2
gpt-5.5	Low	0.0	77.3	77.3	77.3	77.3
	Medium	0.0	0.0	0.0	0.0	0.0
	High	0.0	88.4	0.0	0.0	88.4
gpt-5.6-luna	Low	1.4	0.0	0.0	0.0	1.4
	Medium	0.0	0.0	0.9	0.0	0.9
	High	0.0	0.0	0.0	0.0	0.0
	Max	0.0	0.0	0.0	0.0	0.0
gpt-5.6-sol	Low	88.4	88.4	0.0	0.0	88.4
	Medium	88.4	88.4	88.4	88.4	88.4
	High	88.4	88.4	88.4	88.4	88.4
	Max	0.0	0.0	0.0	88.4	88.4
gpt-5.6-terra	Low	0.0	0.0	0.0	0.0	0.0
	Medium	3.7	0.0	0.0	0.0	3.7
	High	0.0	0.0	3.7	0.0	3.7
	Max	3.7	0.0	0.0	0.0	3.7
gpt-oss-20b	Low	0.0	0.0	0.0	0.0	0.0
	Medium	0.0	0.0	15.7	0.0	15.7
	High	13.0	3.2	3.2	3.2	13.0
gpt-oss-120b	Low	3.2	3.2	3.2	3.2	3.2
	Medium	0.0	0.0	0.0	0.0	0.0
	High	0.0	0.0	0.0	0.0	0.0
minimax-m3	Default	0.0	0.0	0.0	0.0	0.0